W7. Memory Efficient Data Representation

Department of Smart Mobility Engineering
Inwook, Shim
인하대학교
INHA UNIVERSITY

Data has mass.

- The bigger sized data, the slower it moves around.
- Offline storage (e.g., hard-disk or remote cloud storage) run far more slowly to our machine.
- Always Keep in mind your data structure and memory usage.

If you can be parsimonious in your use of RAM, your data will probably get consumed faster, as it'll move around buses faster and more of it will fit into constrained caches.
>> RAM 사용을 절약할 수 있다면, 데이터를 더 빨리 소비할 수 있을 것이다. 왜냐하면 작은 데이터는 버스에서 더 빠르게 이동하고, 한정된 캐쉬 메모리에 사용되기에 더 적합할 것이기 때문이다.

<u>However, It is surprisingly hard to count the amount of RAM used by Python.</u>

Basic Understanding Memory with Python

Python primitive object `list`

- We commonly use primitive object list, storing hundreds or thousands of items.
- A `list` with 100 million (1억) items consumes about 760MB.

  "`list` stores memory address."

```
!pip3 install memory_profiler
%load_ext memory_profiler
%memit [0]*int(1e8)

>> peak memory: 869.10 MiB, increment: 762.76 MiB
```

- 100 million memory address + memory for unique objects

```
%memit [n for n in range(int(1e8))]

>> peak memory: 4949.44 MiB, increment: 4301.18 MiB
```

array module for cheaply storing primitive objects

- The array module efficiently stores primitive types like integers, floats, and characters, but not complex numbers or classes.
- array creates a contiguous block of RAM.

```
import array
%memit array.array('l', range(int(1e8)))

>> peak memory: 2677.72 MiB, increment: 737.66 MiB

array.array('l').itemsize
>> 8
```

762.9394… = 8.0 * (1e8) / 1024**2

array module for cheaply storing primitive objects

- Note that the unique numbers in the array are not Python objects; they are bytes in the array.
- If we were to de-reference any of them, a new Python int object would be constructed.
- That means, if we're going to compute on all data, memory saving result is zero.

해설) 문제는 "역참조(de-reference)": 즉 **array에서 원소를 하나씩 꺼내 다루는 순간!** 원소를 꺼내면 int(...) 같은 **새로운 파이썬 정수 객체**가 만들어짐.
즉, 전체 데이터를 루프를 돌며 하나씩 꺼내 사용한다면, 결국 모든 원소가 파이썬 객체로 "박싱(boxing)" 됨. >> **메모리 절감 효과가 사실상 0**.

! Pass the array to an external process or use only some of the data, you should see a good savings in RAM compared to using a list of integers.

array module for cheaply storing primitive objects

- The array module works with a limited set of datatypes with varying precisions.
- Choose the smallest precision that we need, so that we allocate just as much RAM as needed and no more. It depends on a platform.

```
Type code C Type Minimum size in bytes
'b' signed integer 1
'B' unsigned integer 1
'u' Unicode character 2 (see note)
'h' signed integer 2
'H' unsigned integer 2
'i' signed integer 2
'I' unsigned integer 2
'l' signed integer 4
'L' unsigned integer 4
'q' signed integer 8 (see note)
'Q' unsigned integer 8 (see note)
'f' floating point 4
'd' floating point 8
```

! If we can use numpy, it also good solution to save memory.

Get memory usage of each object directly

- `sys.getsizeof(obj)`
- `int` is a variable-sized object in Python

```
import sys
print(sys.getsizeof(0))
print(sys.getsizeof(1))
print(sys.getsizeof((2**30)-1))
print(sys.getsizeof((2**30)))
```

→ `24,28,28,32` : sets of 4 bytes are added each time the size of the number you're counting steps above the previous limit. This affects only the memory usage; you don't see any difference externally.

python int(PyLongObject)는 가변 길이 정수이며, 64비트 CPython에서는 **정수**를 **30비트씩 잘라서** 저장함.
이 한덩어리를 digit 라고 하며, 하나의 digit가 4 byte (32비트)를 차지함.
메모리 차지량은 객체의 기본헤더(메타데이터)가 24바이트 + 필요한 digit 개수 * 4바이트
size = 24 + 4 *(# of 30-bit digit) byte

0은 특수케이스로 digit를 0개 들고 있음. => 24 Byte
1은 30비트 안에 들어가므로 => 24+4=28Byte
(2**30)-1은 $20^{30} - 1$은 여전히 30비트 안의 숫자 => 28Byte
(2**30)은 30비트를 넘어서는 숫자이므로 1digit 더 필요함. => 24+8=32Byte

Get memory usage of each object directly

- `sys.getsizeof(obj)`
- `int` is a variable-sized object in Python

```
import sys
print(sys.getsizeof(0))
print(sys.getsizeof(1))
print(sys.getsizeof((2**30)-1))
print(sys.getsizeof((2**30)))
```

→ `24,28,28,32` : sets of 4 bytes are added each time the size of the number you're counting steps above the previous limit. This affects only the memory usage; you don't see any difference externally.

- `byte` sequence start with 33 bytes.

```
print(sys.getsizeof(b""))
print(sys.getsizeof(b"a"))
print(sys.getsizeof(b"ab"))
print(sys.getsizeof(b"abc"))
```

→ 33, 34, 35, 36 : An empty byte sequence costs 33 bytes, and each additional character adds 1 byte to the cost.

Get memory usage of each object directly

- `list` is a variable-sized object in Python

```
print(sys.getsizeof([]))
print(sys.getsizeof([1]))
print(sys.getsizeof([1,2]))
```

→ 72, 80, 88 : An empty list costs 72 bytes, and each item(address) in the list takes another 8 bytes on a 64-bit laptop.

sys.getsizeof(list_obj) 는 **컨테이너 자체** (리스트 객체 + 포인터 배열)의 크기를 나타냄.
1, 2 같은 int 객체 자체의 메모리는 미포함

앞과 마찬가지로 객체해더 + 포인터배열, 빈 리스트는 72 Byte (헤더만 포함)
길이 1개는 72+8 = 80 Byte (실제 메모리 사용은 여기에 추가적인 Int 객체 메모리를 차지하고 있음.)

여기선 append함수를 쓸 때 처럼 왜 over-allocation이 일어나지 않나?
>> [1,2,3]처럼 리터럴로 한 번에 만드는 경우에는 보통 정확한 길이만큼만 할당

Get memory usage of each object directly

- `list` is a variable-sized object in Python

```
print(sys.getsizeof([]))
print(sys.getsizeof([1]))
print(sys.getsizeof([1,2]))
```

→ 72, 80, 88 : An empty list costs 72 bytes, and each item in the list takes another 8 bytes on a 64-bit laptop.

- `string in list` (only gives cost for parent object)

```
print(sys.getsizeof([b""]))
print(sys.getsizeof([b"string"]))
print(sys.getsizeof([b"str1", b"str2"]))
```

→ 80, 80, 88 : getsizeof reports only some of the cost, and often just for the parent object.

Get memory usage of each object directly

- `pympler` : trace memory usage as can as possible, but quite slow and cannot count memory behind the scene (e.g., memory usage by wrapping functions of C library.)

```
from pympler.asizeof import asizeof
list_string = [b"str1", b"str2"]
print(asizeof(list_string))          → 168
print(sys.getsizeof(list_string))    → 88
```

- `memit` takes snapshots of the RAM usage reported by the operating system while the statement is executing, and `asizeof` asks the objects about their size (which may not be reported correctly).

Sparse Matrix

Sparse Matrix

$$\begin{pmatrix} 1.0 & 0 & 5.0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3.0 & 0 & 0 & 0 & 0 & 11.0 & 0 \\ 0 & 0 & 0 & 0 & 9.0 & 0 & 0 & 0 \\ 0 & 0 & 6.0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 7.0 & 0 & 0 & 0 & 0 \\ 2.0 & 0 & 0 & 0 & 0 & 10.0 & 0 & 0 \\ 0 & 0 & 0 & 8.0 & 0 & 0 & 0 & 0 \\ 0 & 4.0 & 0 & 0 & 0 & 0 & 0 & 12.0 \end{pmatrix}$$

- A matrix in which most elements are 0.
- In the viewpoint of memory savings,

  many algorithms have special ways of dealing with sparse matrices.

Matrix Market, https://math.nist.gov/MatrixMarket/
Harwell-Boeing sparse matrix collection, http://swmath.org/software/8516
SuiteSparse Matrix Collection, https://sparse.tamu.edu/
UF Sparse Matrix Collection, https://www.cise.ufl.edu/research/sparse/ matrices/list_by_id.html

Sparse Matrix : COO(coordinate) the simplest implementation

- A matrix in which most elements are 0.
- In the viewpoint of memory savings,

  many algorithms have special ways of dealing with sparse matrices.

$$A = \begin{bmatrix} 0 & 1 & 2 & 0 \\ 3 & 4 & 0 & 0 \\ 0 & 0 & 5 & 0 \\ 6 & 0 & 0 & 0 \end{bmatrix}$$

```
Value: [1 2 3 4 5 6]
Row  : [0 0 1 1 2 3]
Col  : [1 2 0 1 2 0]
```

```python
from scipy import sparse
import numpy as np

data = np.array([1,2,3,4,5,6])
row  = np.array([0,0,1,1,2,3])
col  = np.array([1,2,0,1,2,0])

sparse_coo = sparse.coo_matrix((data, (row,col)))

print(type(sparse_coo))
print(sparse_coo)
dense_mat=sparse_coo.toarray()
print(type(dense_mat))
print(dense_mat)
```

Sparse Matrix : COO the simplest implementation

- A matrix in which most elements are 0.
- In the viewpoint of memory savings,

  many algorithms have special ways of dealing with sparse matrices.

$$A = \begin{bmatrix} 0 & 1 & 2 & 0 \\ 3 & 4 & 0 & 0 \\ 0 & 0 & 5 & 0 \\ 6 & 0 & 0 & 0 \end{bmatrix}$$

```
Value: [1 2 3 4 5 6]
Row  : [0 0 1 1 2 3]
Col  : [1 2 0 1 2 0]
```

! We need three numbers for each elements.
→ As long as our matrix has at least 66% zero entries, we can reduce memory.

COO matrices are generally used only to construct sparse matrices, not for computation.

Sparse Matrix : CSR(Compressed Sparse Row) implementation

$$A = \begin{bmatrix} 0 & 1 & 2 & 0 \\ 3 & 4 & 0 & 0 \\ 0 & 0 & 5 & 0 \\ 6 & 0 & 0 & 0 \end{bmatrix}$$

COO

```
Value: [1 2 3 4 5 6]
Row  : [0 0 1 1 2 3] >> [0 2 4 5 6]
Col  : [1 2 0 1 2 0]
```

start_index

Number of elements
(end of index +1)

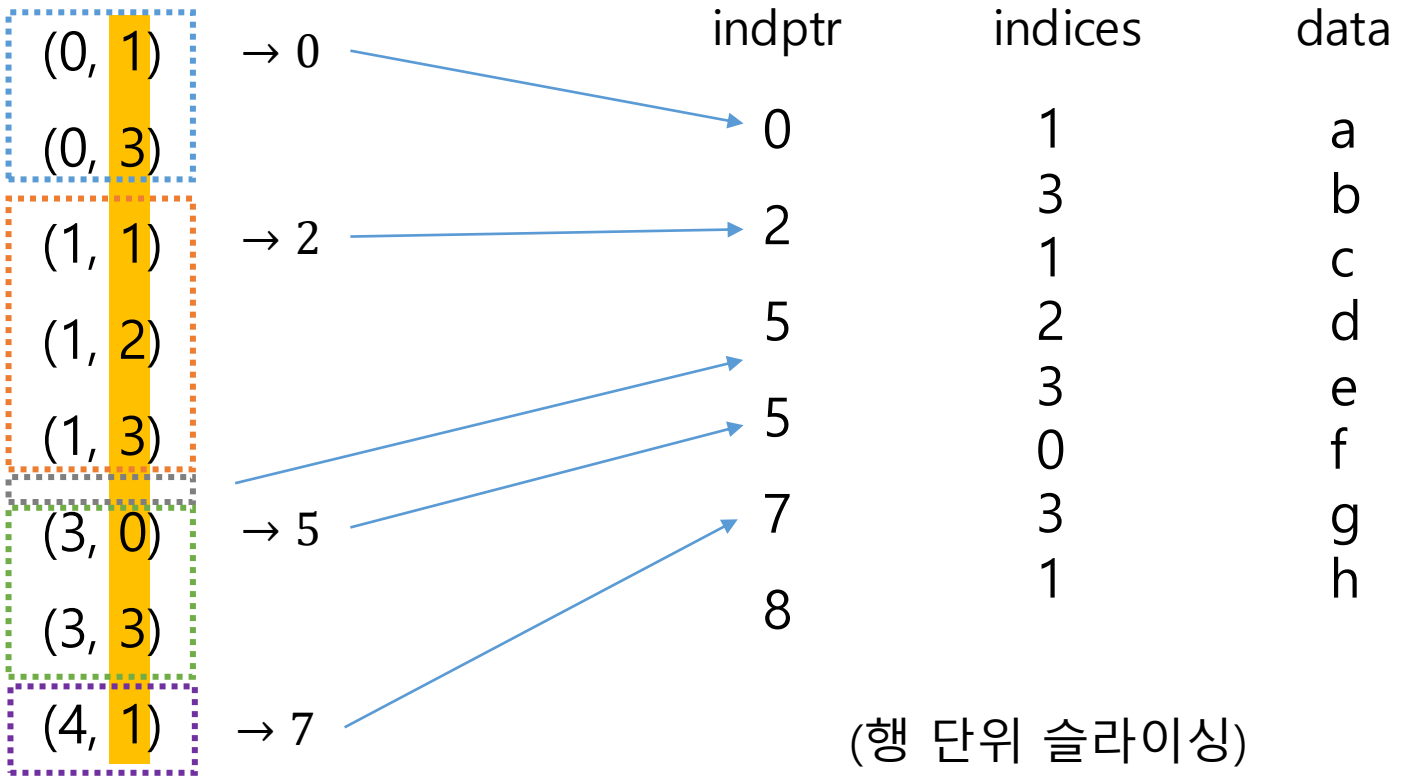
```python
data = np.array([1,2,3,4,5,6])
row = np.array([0,2,4,5,6])
col = np.array([1,2,0,1,2,0])

sparse_csr = sparse.csr_matrix((data, col, row))

print(type(sparse_csr))
print(sparse_csr)
dense_mat=sparse_csr.toarray()
print(type(dense_mat))
print(dense_mat)
```

Sparse Matrix : CSR(Compressed Sparse Row) implementation

$$A = \begin{bmatrix} 0 & a & 0 & b \\ 0 & c & d & e \\ 0 & 0 & 0 & 0 \\ f & 0 & 0 & g \\ 0 & h & 0 & 0 \end{bmatrix}$$



(행 단위 슬라이싱)

Sparse Matrix : CSR(Compressed Sparse Row) implementation

- Sparse Matrix-Vector Multiplication

$y = Ax$, sparse matrix A and vector x

$$A = \begin{bmatrix} 4 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 2 \\ 5 & 0 & 0 & 0 \end{bmatrix} \qquad x = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$$

data = [4,1,3,2,5]
indices = [0,2,1,3,0] (각 값의 열 인덱스)
indptr = [0,2,2,4,5] (행 시작 포인터)

$$y_i = \sum_{k=indptr[i]}^{indptr[i+1]-1} data[k] \cdot x_{indices[k]}$$

Example) $y_0 = \sum_{k=indptr[0]}^{indptr[1]-1} A[k] \cdot x_{indices[k]}$

$$y_0 = \sum_{k=0}^{1} data[k] \cdot x_{indices[k]}$$

$$= data[0] \cdot x_0 + data[1] \cdot x_2 = 4 \cdot 1 + 1 \cdot 3 = 7$$

- SpMM (Sparse Matrix-Matrix Multiplication) $Y = AX, where\ X \in \mathbb{R}^{n \times d}$

(실제로는 한번에 처리하지만 개념적으로는 열마다 SpMV를 d 번 수행)

Sparse Matrix : more …

- Diagonal-wise storage scheme

$$A = \begin{bmatrix} \cdot & a_{01} & \cdot & \cdot \\ a_{10} & a_{11} & a_{12} & \cdot \\ \cdot & a_{21} & a_{22} & a_{23} \\ \cdot & \cdot & a_{32} & a_{33} \end{bmatrix}, \quad \text{offsets} = [-1,\ 0,\ +1]$$

```
data (D × n), offsets = [-1, 0, +1]

k = -1  →  data[0, j] = A[j+1, j]  →  [ a10   a21   a32    ·   ]
k =  0  →  data[1, j] = A[j,   j]  →  [ a00   a11   a22   a33 ]
k = +1  →  data[2, j] = A[j-1, j]  →  [  ·    a01   a12   a23 ]
                                          j=0   1     2     3
```

+밴드형/규칙형 희소성(tridiagonal, pentadiagonal, stencil)에서 최소 메모리 + 최고 SpMV 성능.

+연속 메모리(대각별 1D)라 SIMD/캐시 적중률이 매우 높음.

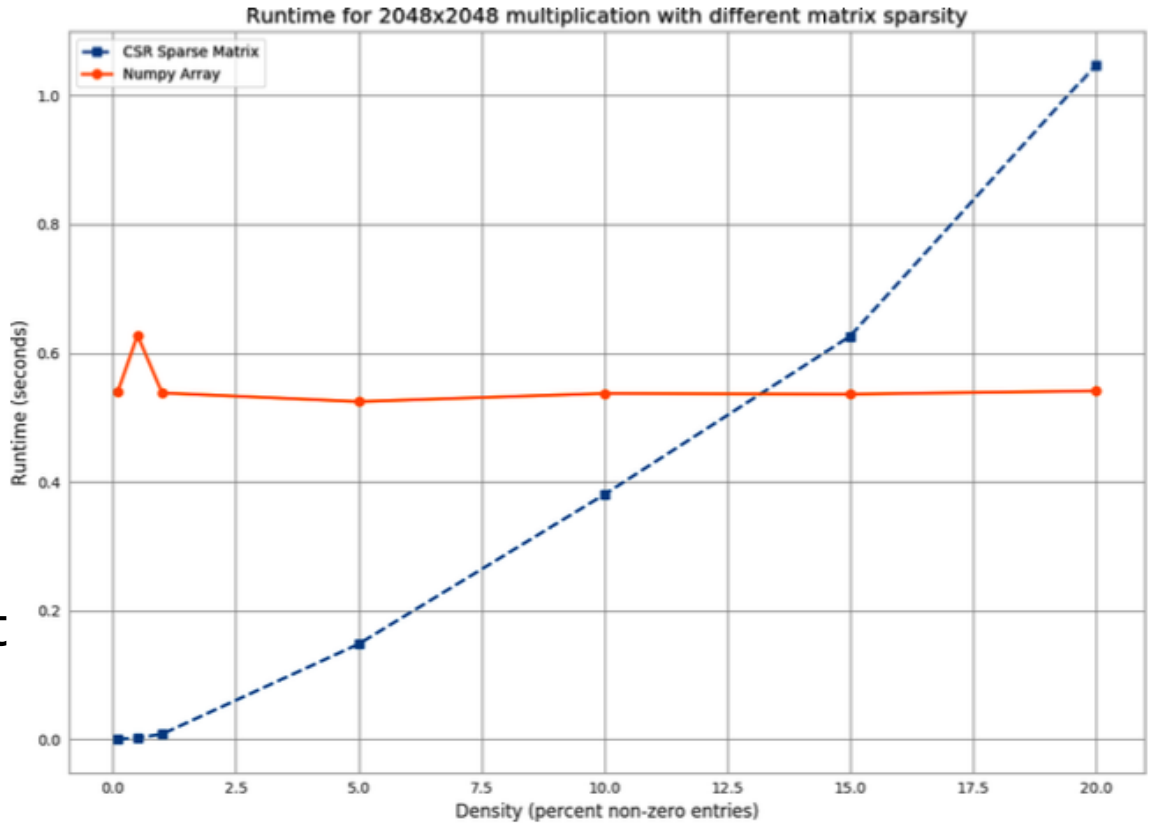
+행/열 인덱스 저장(예: CSR/CSC의 indices)이 거의 불필요하여 오버헤드 낮음

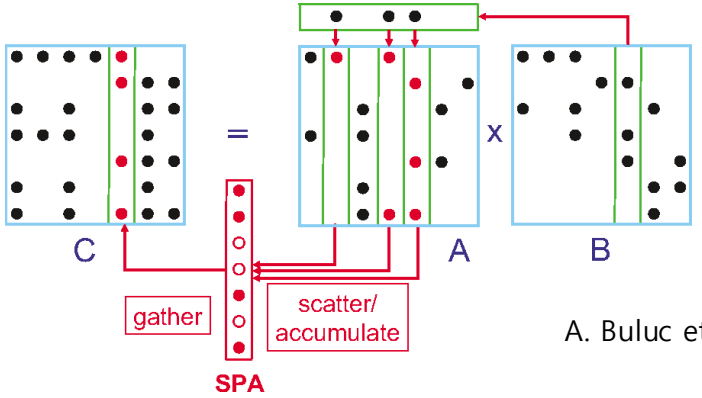
- 불규칙 희소성(무작위/그래프형)에는 패딩 낭비가 심함
- 삽입/구조 변경이 불편(대각 집합이 바뀌면 배열 재구성).

Computation

- Running only the multiplication operation for elements that are non-zero (thus reducing the number of operations needed), we also don't need to allocate such a large amount of space to save our result in.

- It is a balance between losing the use of efficient caching and vectorization versus not having to do a lot of the calculations associated with the zero values of the matrix.



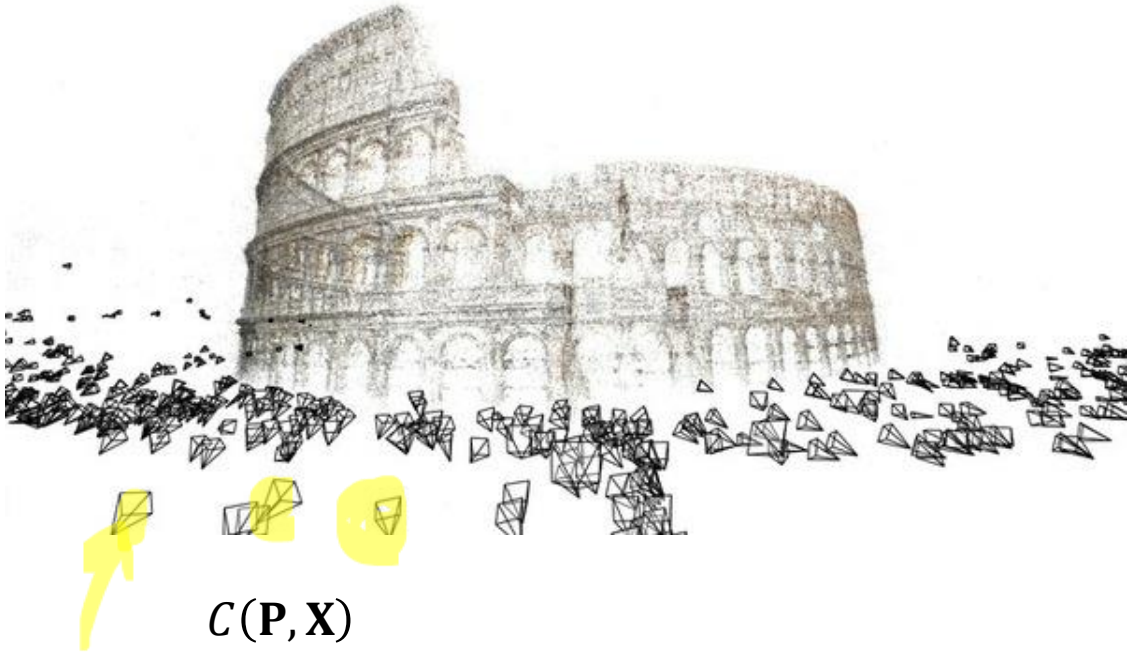


A. Buluc et al., "Implementing Sparse Matrices for Graph Algorithms," Computer Science 2011

Limitations of sparse matrix

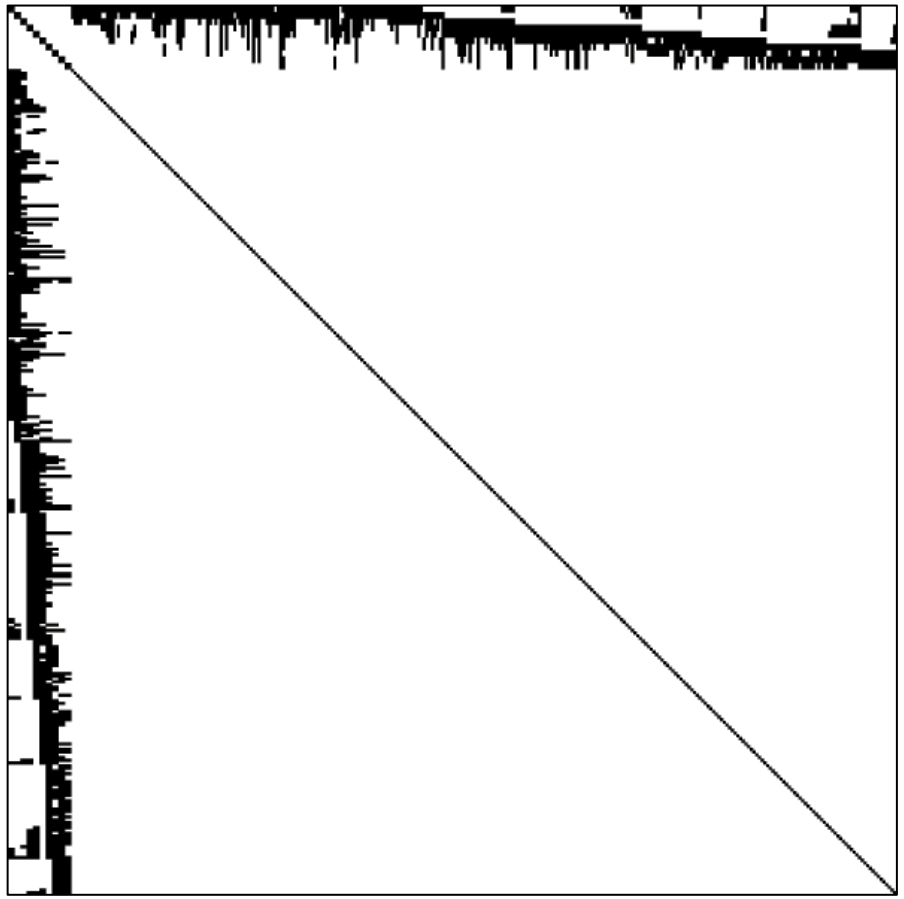
- The amount of support for sparse matrices is quite low. https://en.wikipedia.org/wiki/Sparse_matrix
- Understanding which is the best one to use and when to use it demands some expert knowledge.

Examples of Sparse Matrix : SfM



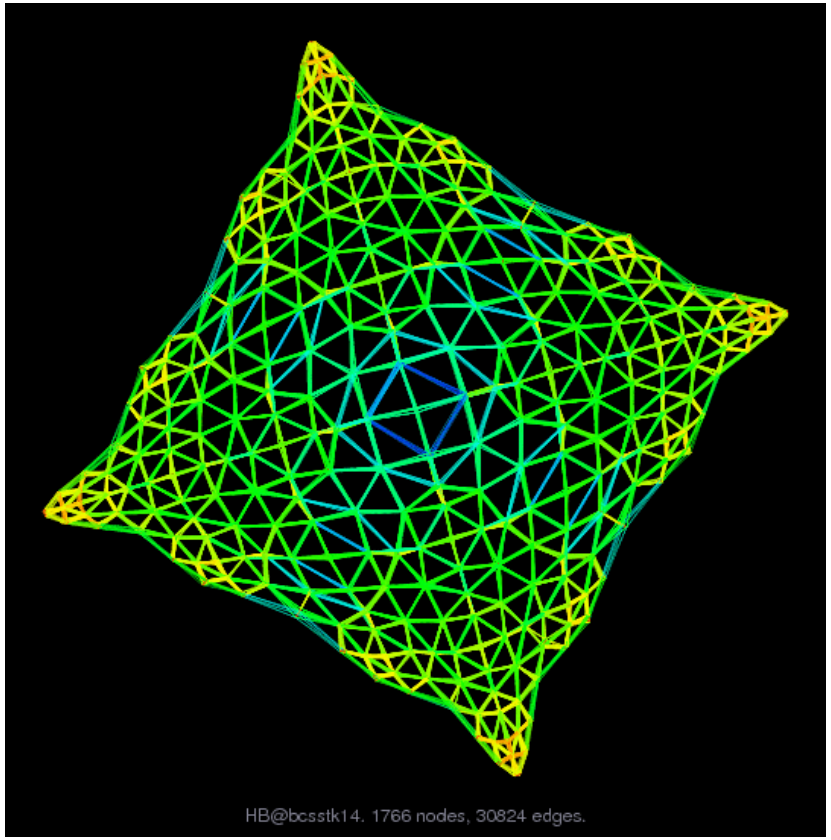
$$C(\mathbf{P},\mathbf{X}) = \sum_{i=1}^{n}\sum_{j=1}^{m} w_{ij} \parallel \mathbf{u}_{ij} - \langle \mathbf{K}\mathbf{P}_i\mathbf{X}_j \rangle \parallel^2$$

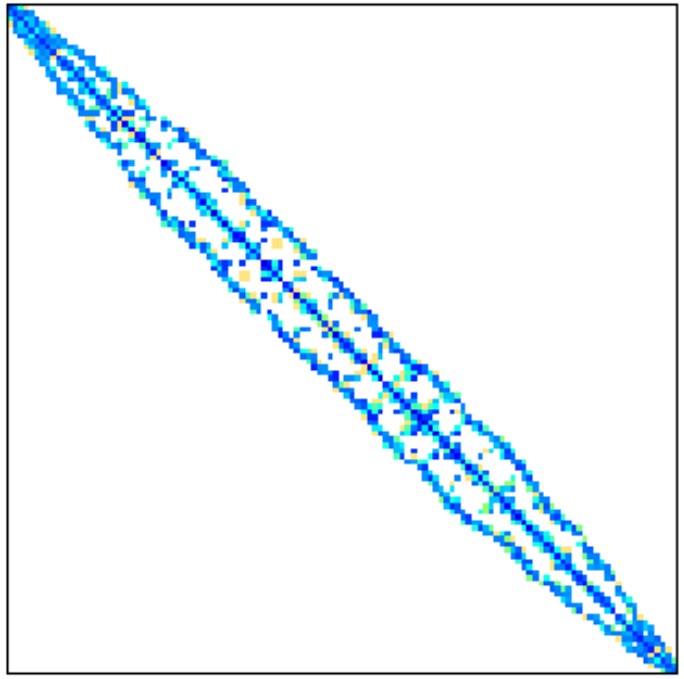
n: The number of cameras
m: The number of features
$\pi(\cdot)$: The projection function
$(\mathcal{R}^3 \rightarrow \mathcal{R}^2)$
w_{ij}: indicator variable
1 if visible, 0 other wise



A sparse matrix obtained when solving a modestly sized bundle adjustment problem.

Examples of Sparse Matrix : Mesh





https://www.cise.ufl.edu/research/sparse/matrices/HB/bcsstk14.html

Examples of Sparse Matrix : Graph

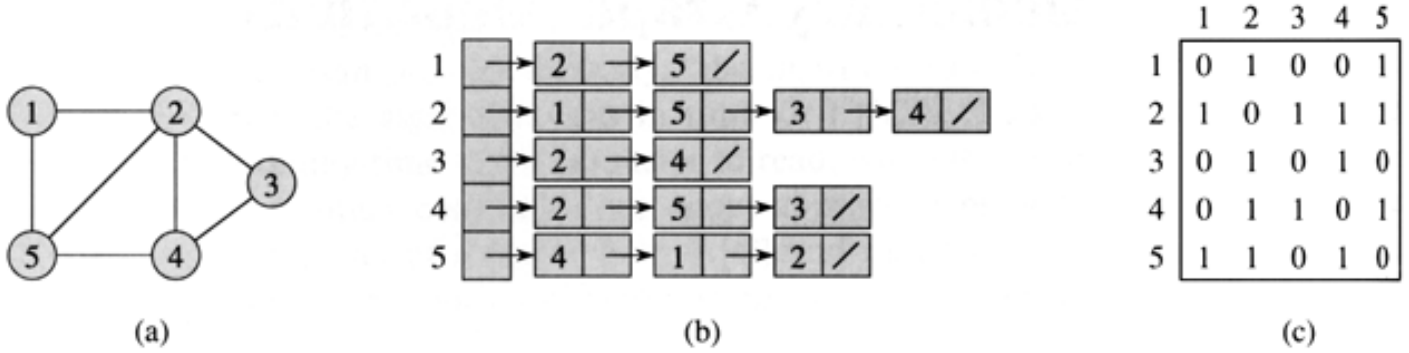


Figure 23.1 Two representations of an undirected graph. **(a)** An undirected graph G having five vertices and seven edges. **(b)** An adjacency-list representation of G. **(c)** The adjacency-matrix representation of G.

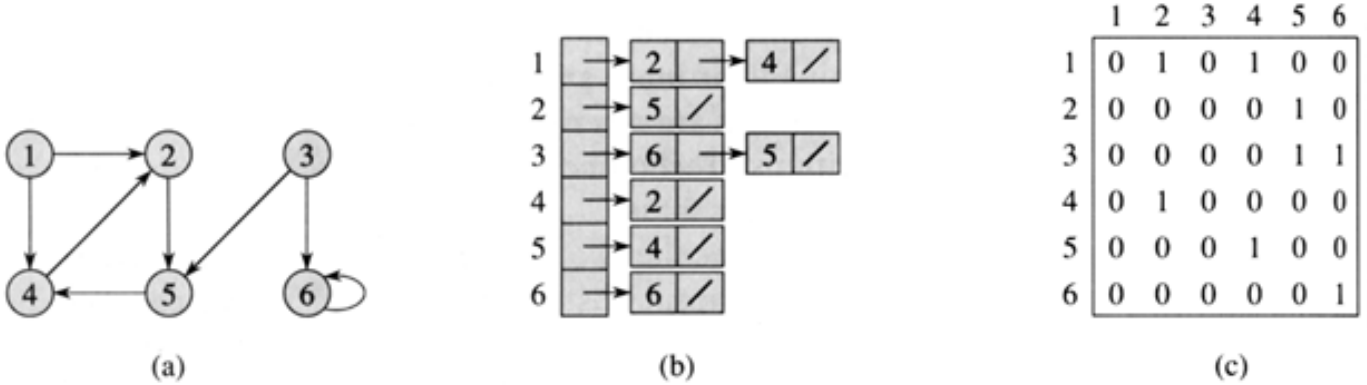
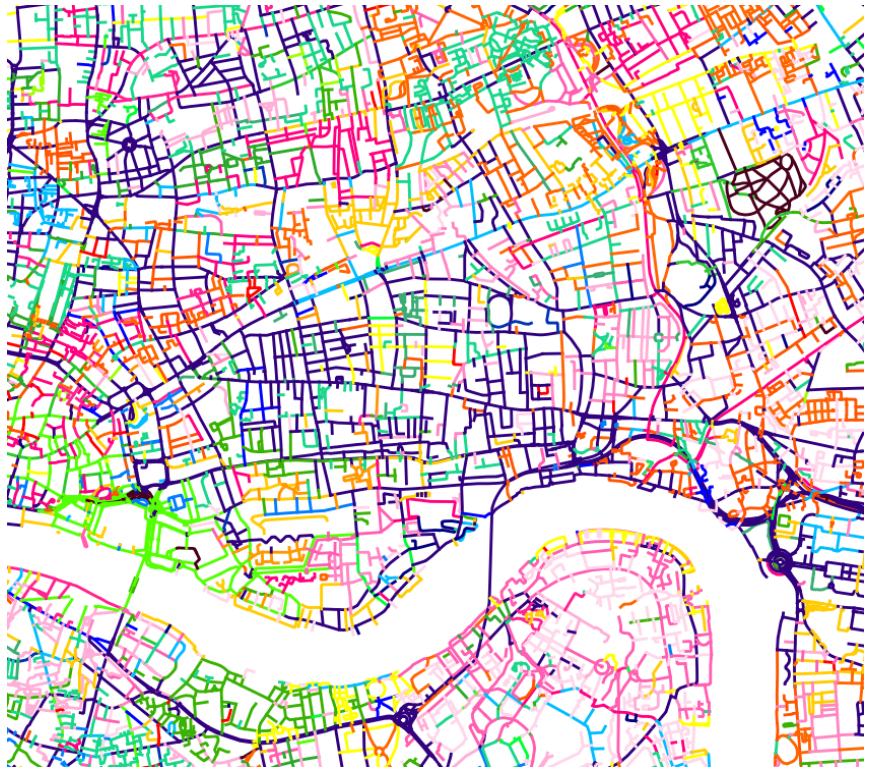


Figure 23.2 Two representations of a directed graph. **(a)** A directed graph G having six vertices and eight edges. **(b)** An adjacency-list representation of G. **(c)** The adjacency-matrix representation of G.

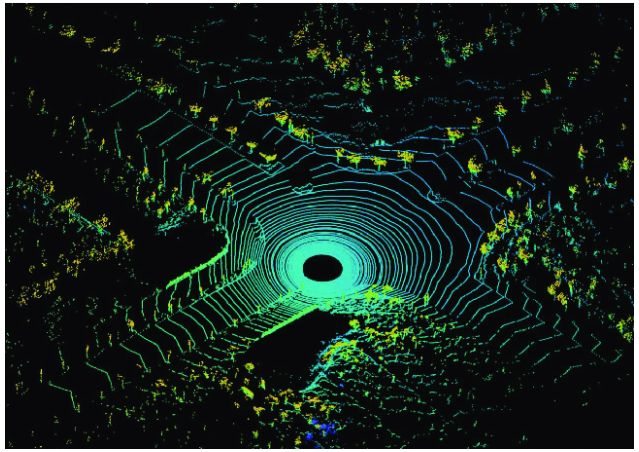


??

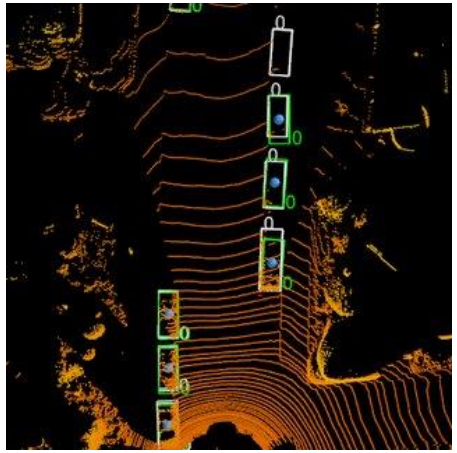
https://www.cs.fsu.edu/~burmeste/slideshow/chapter23/23-1.html

Map Representation

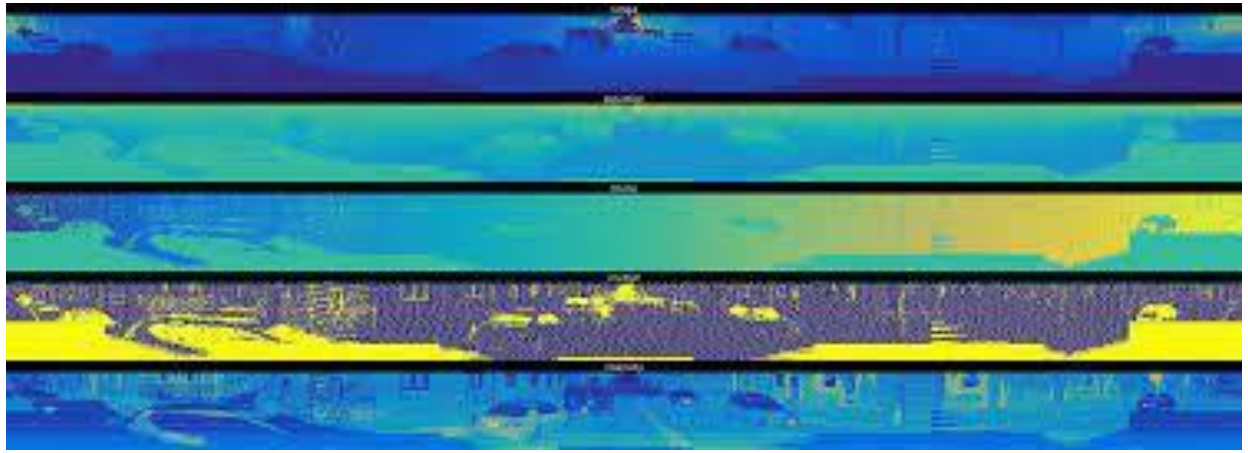
3D Information



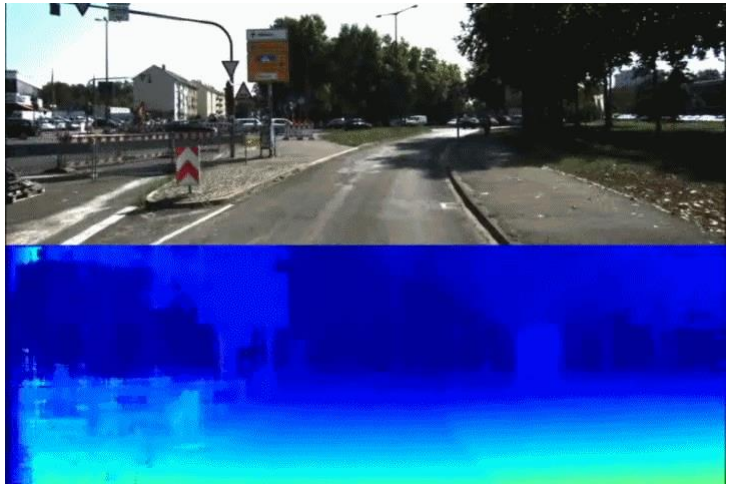
3D LiDAR (64 layers)



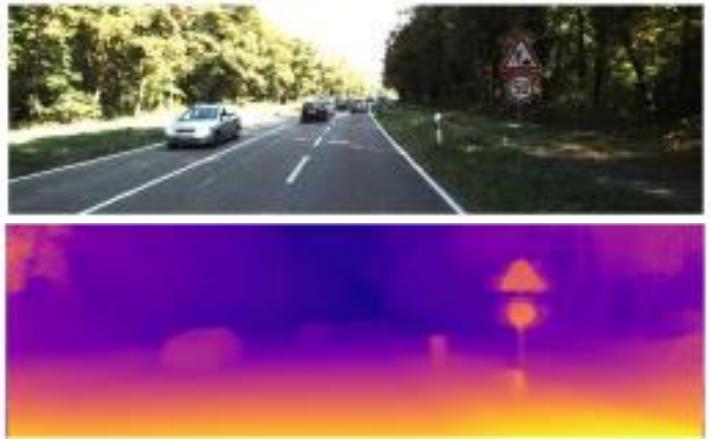
BEV



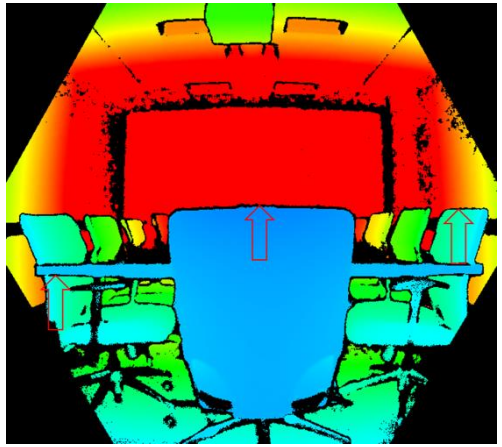
Range Image



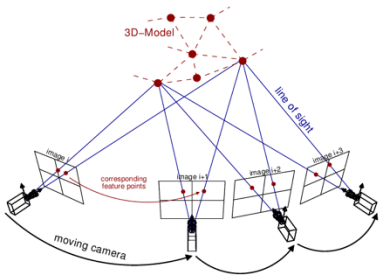
Stereo Matching



Single Image Depth



Indoor Depth Camera



SfM*

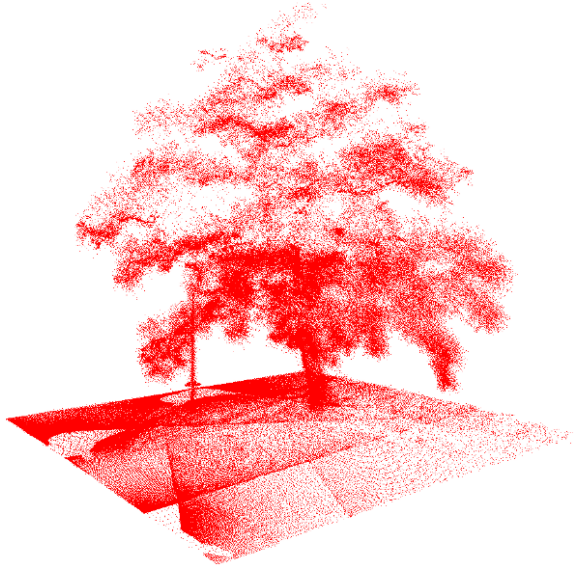
Structure from Motion

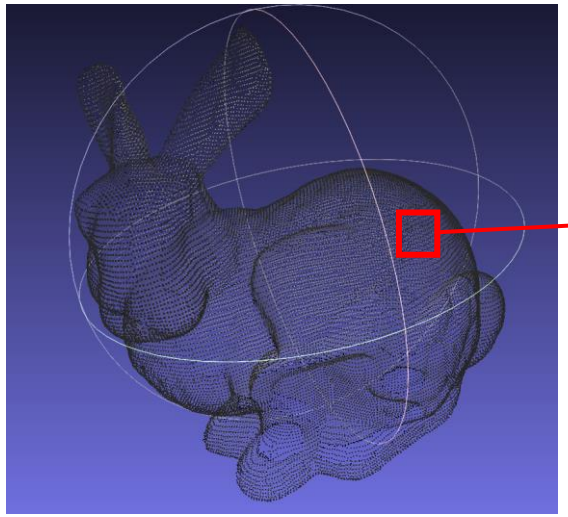
Point Clouds

Pros:
- No discretization of data (no information loss).
- Mapped area is not limited.

Cons:
- Unbounded memory usage
- No direct representation of FREE or UNKNOWN space.

  >> Volumetric map representation is required.

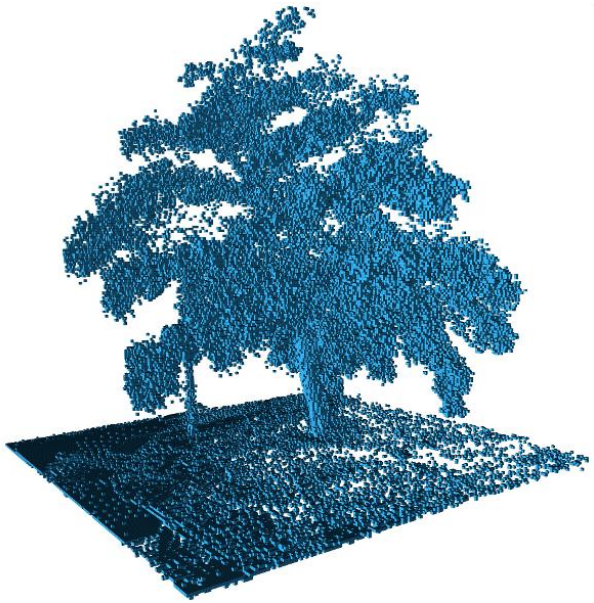




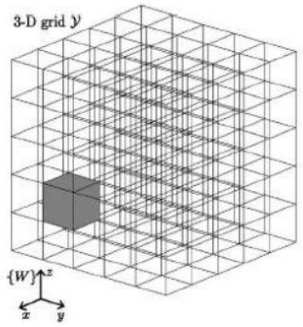
	1	2	3	4
1	−0.0378	0.1279	0.0045	
2	−0.0448	0.1289	0.0019	
3	−0.0680	0.1512	0.0372	
4	−0.0023	0.1302	0.0232	
5	−0.0226	0.1267	0.0072	
6	−0.0251	0.1259	0.0062	
7	−0.0371	0.1274	0.0018	
8	0.0332	0.1127	0.0277	
9	0.0380	0.1098	0.0162	
10	−0.0255	0.1126	0.0367	
11	−0.0245	0.1126	0.0373	
12	0.0274	0.1216	0.0212	
13	−0.0629	0.1584	−0.0176	
14	0.0401	0.1042	0.0222	
15	0.0452	0.0932	0.0112	
16	−0.0325	0.1742	−0.0024	
17	−0.0805	0.1358	0.0500	
18	−0.0725	0.1260	0.0529	
19	−0.0804	0.1372	0.0491	
20	0.0080	0.1313	0.0169	
21	0.0400	0.1070	0.0082	
22	0.0372	0.0997	−0.0098	
23	−0.0906	0.1406	0.0222	
24	−0.0843	0.1090	0.0244	
25	−0.0689	0.1563	0.0148	
26	−0.0845	0.1523	0.0270	
27	0.0054	0.1258	0.0318	
28	0.0089	0.1244	0.0331	
29	−0.0315	0.1258	0.0039	
30	−0.0497	0.1417	0.0124	
31	−0.0785	0.1359	0.0506	
32	−0.0043	0.1280	0.0283	
33	−0.0455	0.1234	0.0251	
34	−0.0235	0.1127	0.0380	
35	−0.0790	0.0936	−0.0106	
36	0.0236	0.1242	0.0226	

3D Voxel Grids

Pros:
- Volumetric representation
- Constant access time
  >> each voxel has their absolute space in a coordinate.
- Probabilistic update
  >> We usually use Baye's rule for the update voxel.



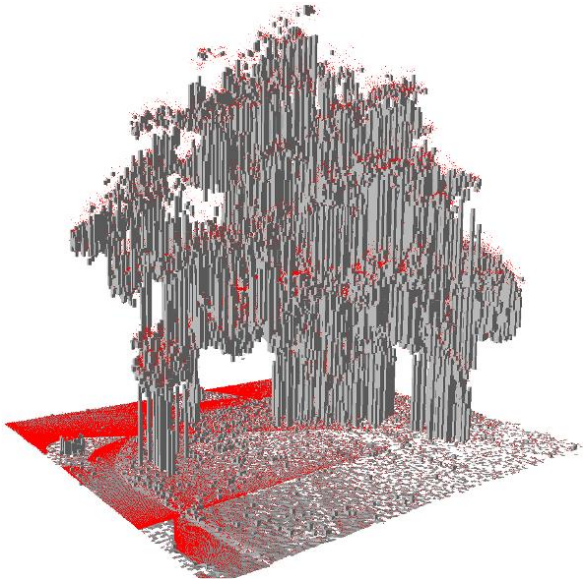
Cons:
- Large memory usage: Complete map is allocated in memory.
  >> We must handle empty voxel in memory.
- We should prepare the map extension in advance.



Elevation Map

Average over all scan points that fall into a cell.
  * only single height information exists in single cell.
  ** This is why we call the elevation map as 2.5D map.

Pros:
- Memory efficient (We need only 2D grid cell.)
- Constant access time
  >> each voxel has their absolute space in a coordinate.

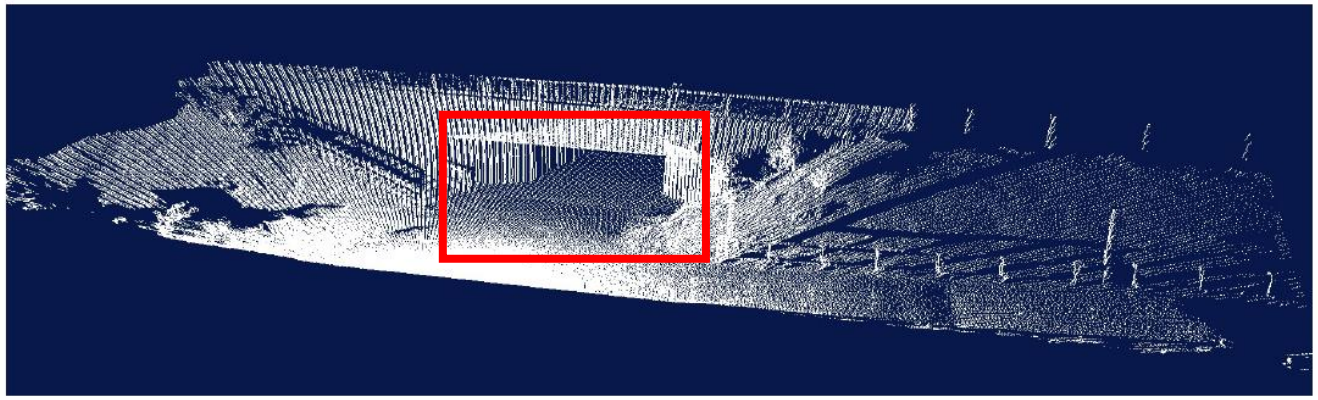


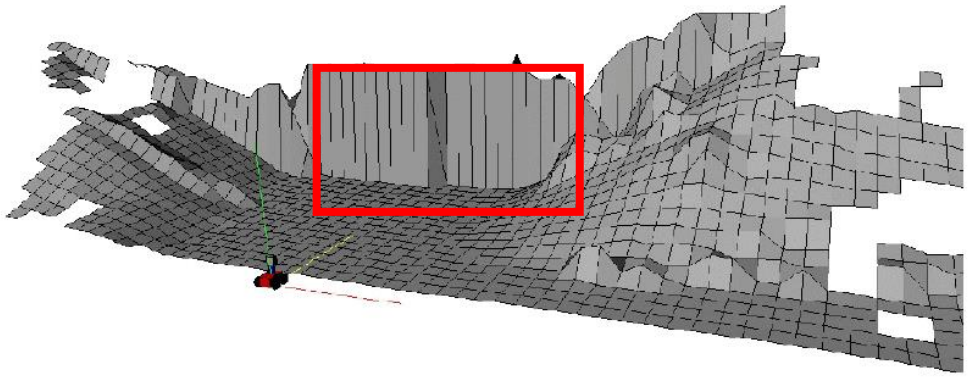
Cons:
- No distinction between free and unknown space.
  >> e.g., tunnel
- Only one level is representation.
- We should prepare the map extension in advance.

Elevation Map (cont.)

2D grid that stores an estimated height for each cell.

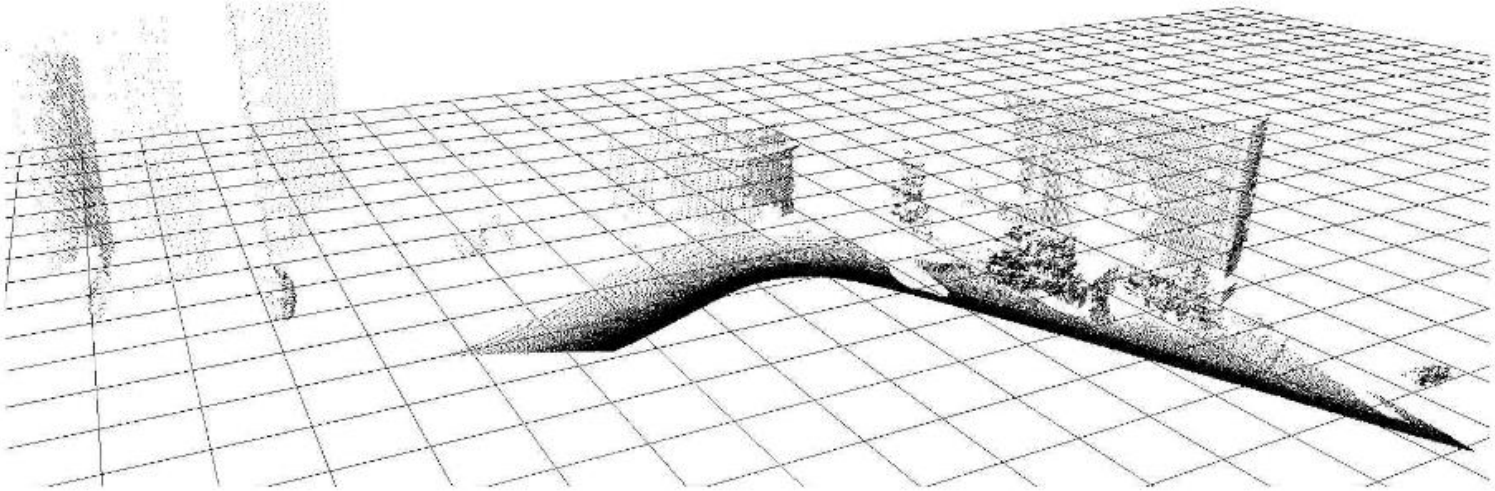
- Uncertainty increases as the measuring distance increases.
- Kalman update to estimate the elevation



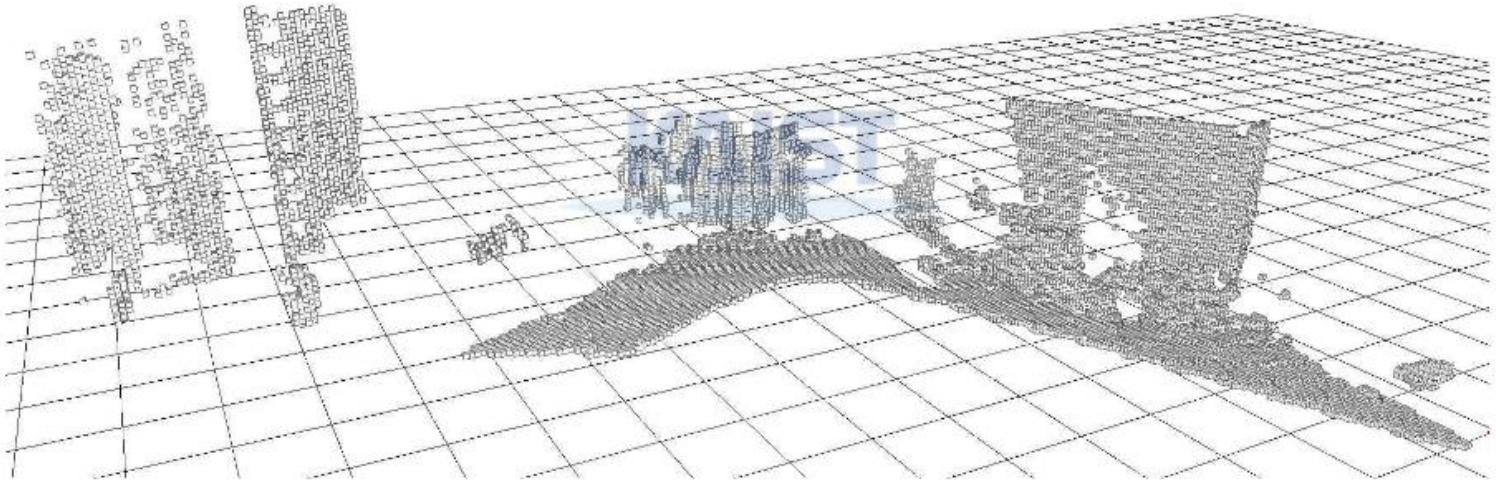


Images from Wolfram Burgard

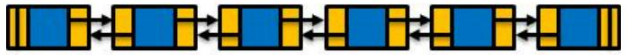
Multi-link Feature Voxel



(a) Oakland Point Cloud Scene #2



(b) Result of converting point cloud to voxel



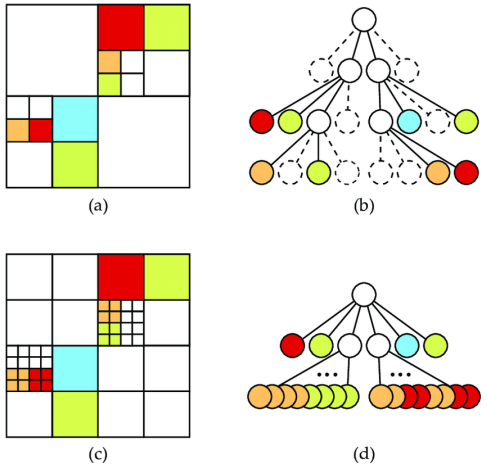
Double Linked List

Oakland #1	Point Cloud	fVoxel
Number	100,000	11,315
Memory	1.14MB	144KB
Compression	0.123	
Range	109.65m(W) x 77.9m(D) x 34.6m(H)	

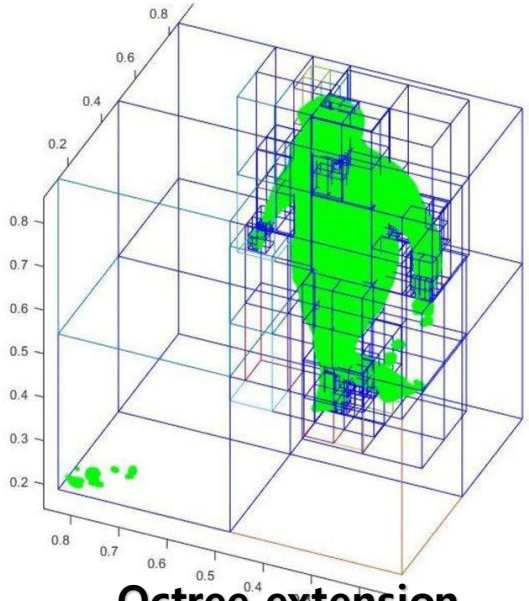
$$X \times Y \times A$$
$$\times datasize$$

A: adaptive size

Octree

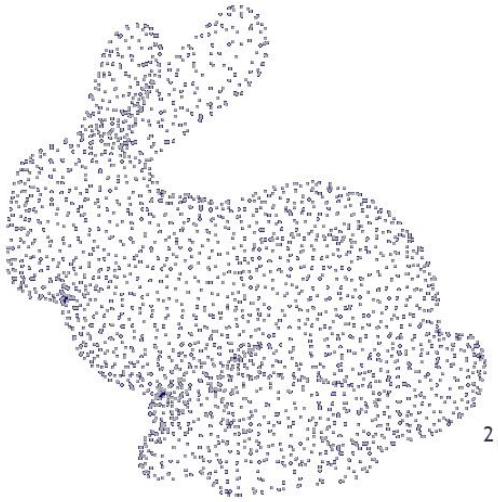


Graph Structure for quadtree

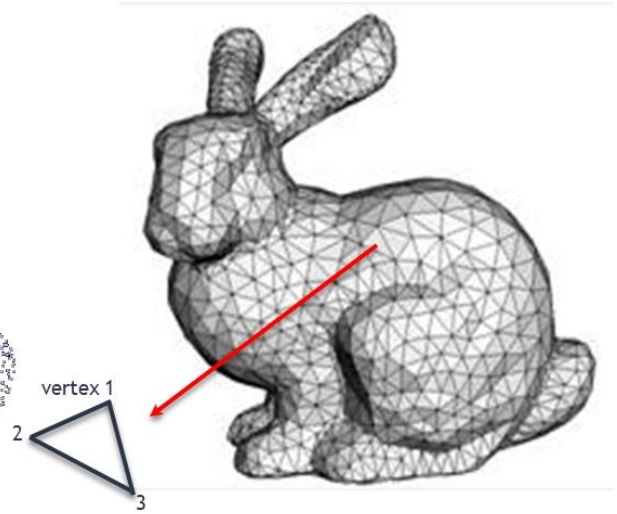


Octree extension

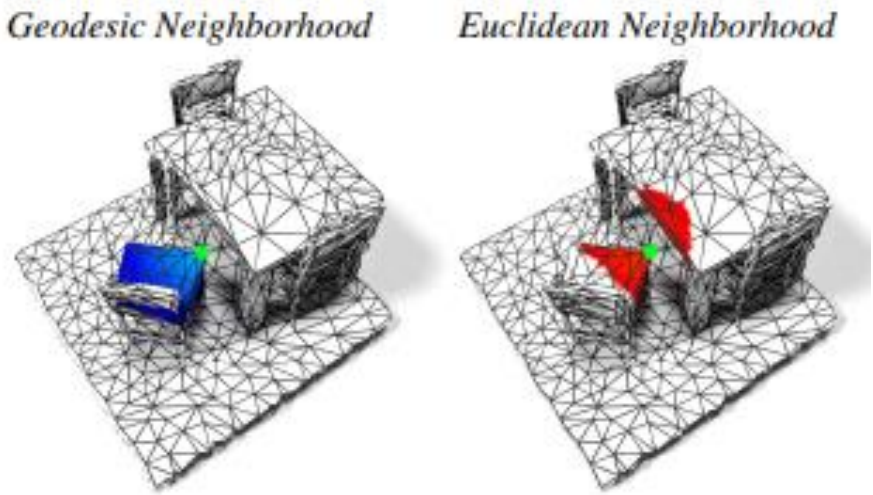
Mesh

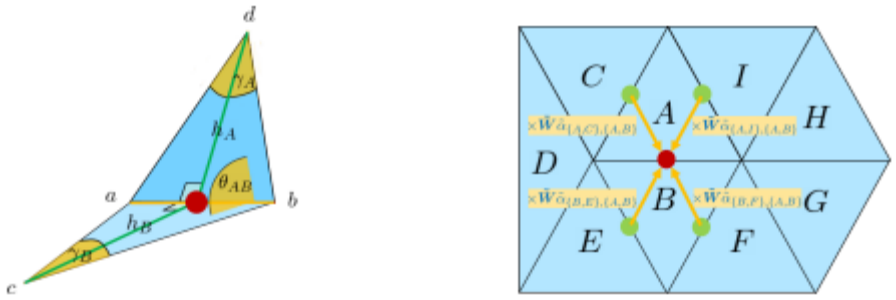


Point Cloud

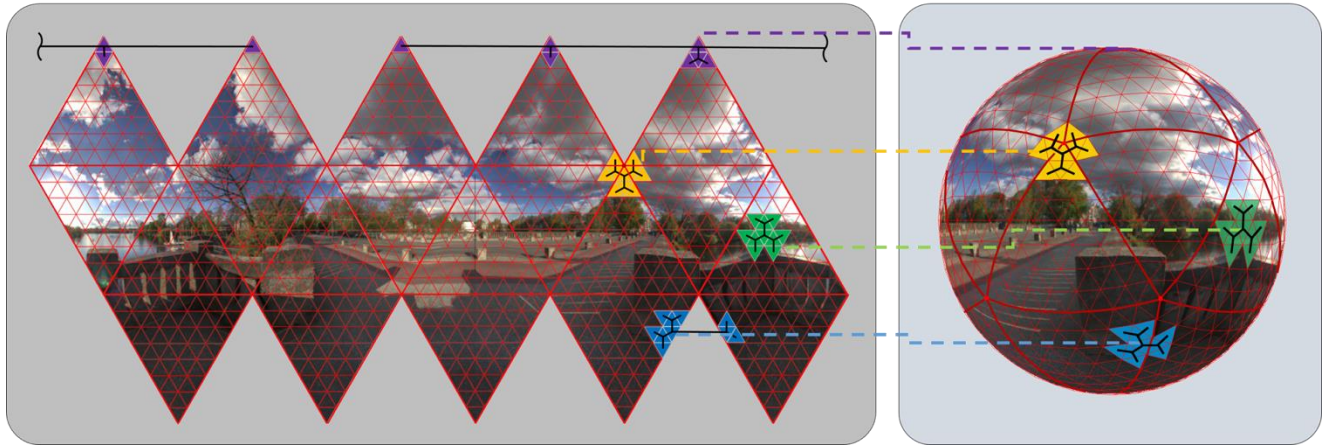


Mesh (shaded)



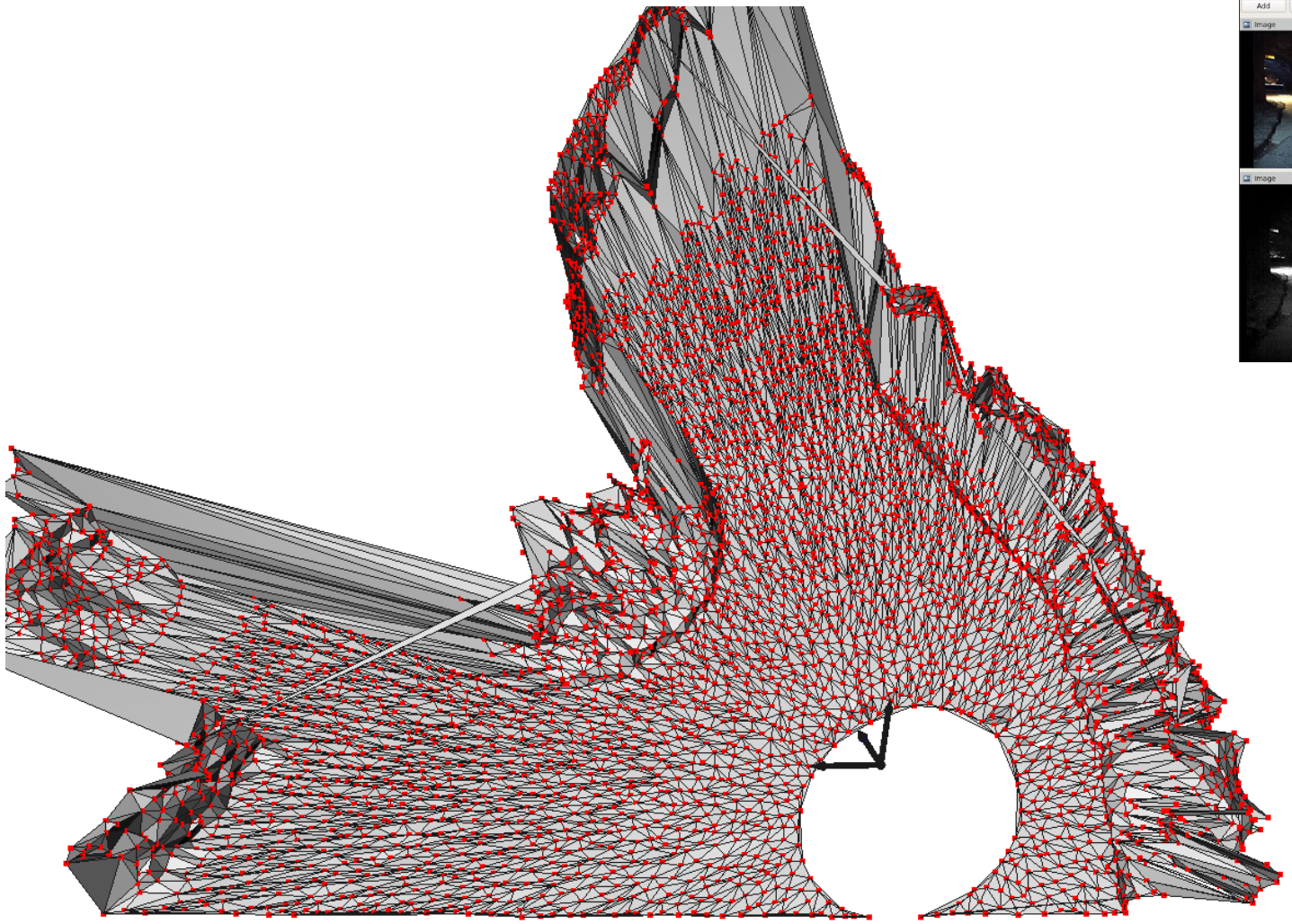


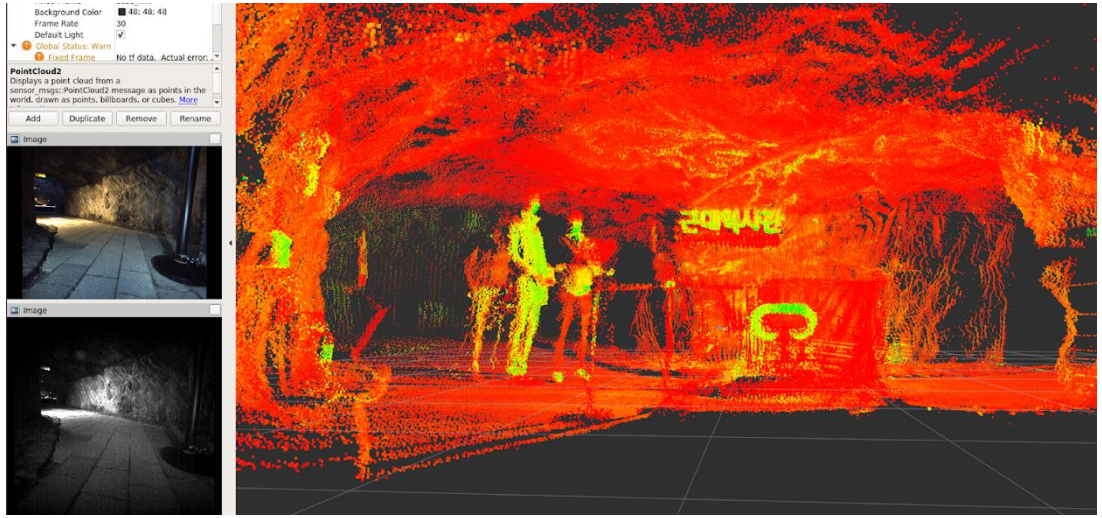
Dual Mesh Convolution

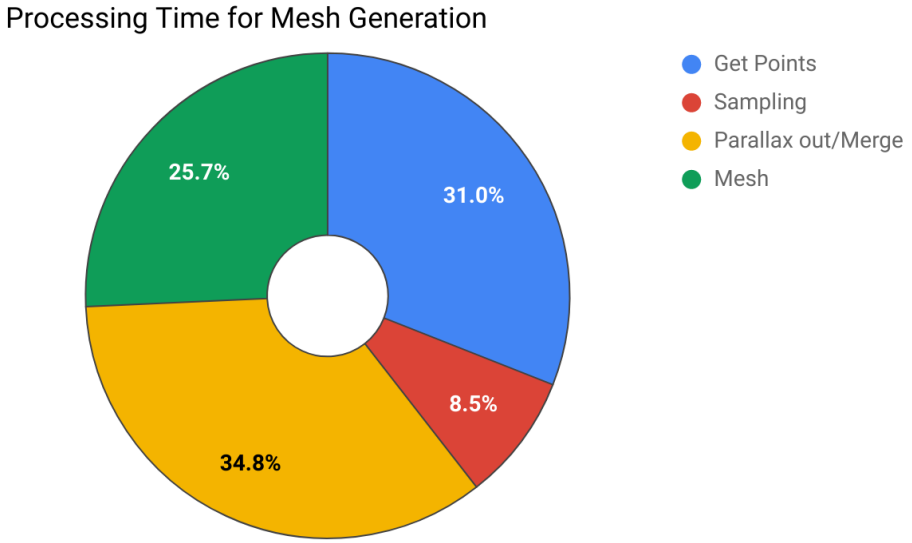


Mesh for 360 degree Images

Mesh (cont.)




Background Color
Frame Rate
Default Light
Global Status: Warn
Fixed Frame
No tf data. Actual error...
PointCloud2
Displays a point cloud from a sensor_msgs::PointCloud2 message as points in the world, drawn as points, billboards, or cubes. More
Add
Duplicate
Remove
Rename
Image
Image


Processing Time for Mesh Generation
31.0%
8.5%
34.8%
25.7%
Get Points
Sampling
Parallax out/Merge
Mesh

One more thing

NumExpr

- Large vectorized expressions in NumPy can create intermediate large arrays during complex operations.
- These occur invisibly and may draw attention to themselves only when an out-of-memory error occurs.

```
!pip3 install ipython_memory_usage
import ipython_memory_usage.ipython_memory_usage as imu
import numpy as np
%ipython_memory_usage_start
'memory profile enabled'
```

NumExpr

```
[2]  1 num_items = 100_000_000
     2 pred = np.random.uniform(low=0.0000001, size=num_items)
```

In [2] used 764.0859 MiB RAM in 1.32s, peaked 0.00 MiB above current, total RAM usage 860.78 MiB

```
[3]  1 gt = np.ones(shape=num_items)
```

In [3] used 763.5234 MiB RAM in 0.43s, peaked 0.00 MiB above current, total RAM usage 1624.30 MiB

```
[4]  1 eval = -(gt * np.log(pred) + ((1-gt)) * (np.log(1-pred)))
```

In [4] used 3054.8086 MiB RAM in 7.72s, peaked 0.00 MiB above current, total RAM usage 4679.11 MiB

```
1 import numexpr
2 eval = numexpr.evaluate("-(gt * log(pred) + ((1-gt)) * (log(1-pred)))")
```

In [5] used 763.9023 MiB RAM in 7.99s, peaked 0.00 MiB above current, total RAM usage 2388.79 MiB

+ 코드 + 텍스트